

ACKNOWLEDGEMENT

I would like to express my deepest gratitude and appreciation to everyone who contributed to the development and success of this chatbot project.

First and foremost, I am immensely thankful to my supervisor, **SHEKH TANJIL SHARIF, and Head of the Department SK. MAMUNUR RASHID, INSTITUTE OF SCIENCE TRADE AND TECHNOLOGY**, for the facilities provided to accomplish this main project.

I feel elated in manifesting our sense of gratitude to our internal project guide **SHEKH TANJIL SHARIF, Department of Computer Science and Engineering**, Institute of Science Trade and Technology. He has been a constant source of inspiration for me, and I am very deeply thankful to him for his support and valuable advice.

I extend my heartfelt thanks to all the developers and engineers who worked tirelessly to design, implement and refine the chatbot's functionalities. Your technical skills, attention to detail, and commitment to excellence were instrumental in creating a robust and user-friendly chatbot.

I am extremely grateful to our Departmental staff members, Lab technicians, and non-teaching staff members for their extreme help throughout our project.

Finally, I would like to thank my friends, family, and loved ones for their unwavering support and encouragement throughout this project.

Thank you all once again for your invaluable contributions and support.

Sincerely,

RAKIBUR RAHMAN (18502003671)

DECLARATION

I hereby declare that project titled **“Simple Chatbot for University Admission Help”** is an original record done by me at **ISTT** towards the partial fulfillment of requirement for the award of degree of Bachelor of Science in Computer Science and Engineering during the period of 2018-19 in **ISTT**, and I state that this project has not been submitted anywhere in the partial fulfillment for any degree of this or any other University.

RAKIBUR RAHMAN (18502003671)

ABSTRACT

This project aims to develop a simple chatbot capable of engaging in natural language conversations with users. Chatbots have gained significant popularity in various domains, including customer support, information retrieval, and personal assistance. The objective of this project is to create an efficient and user-friendly chatbot that can understand and respond to user queries in a conversational manner.

In this Python project with source code, we are going to build a chatbot using deep learning techniques. The chatbot will be trained on the dataset which contains categories (intents), patterns, and responses. We use a special recurrent neural network (LSTM) to classify which category the user's message belongs to and then we will give a random response from the list of responses.

Keywords: chatbot, natural language processing, machine learning, conversational AI, dialogue management.

INDEX

1. INTRODUCTION

- 1.1 Project Background
- 2.2 Scope of the Project

2. ANALYSIS

- 2.1 Existing system and proposed system
 - 2.1.1 History and Evolution of the system
 - 2.1.2 Proposed System
- 2.2 Feasibility Study
 - 2.2.1 Economic Feasibility
 - 2.2.2 Technical Feasibility
 - 2.2.3 Operational Feasibility
- 2.3 Nonfunctional Requirements
 - 2.3.1 Product Requirements
 - 2.3.2 Organizational Requirements
- 2.4 Functional Requirement
 - 2.4.1 User
 - 2.4.2 Amin
 - 2.4.3 Product
- 2.5 Project Requirement Specification
 - 2.5.1 Hardware Requirements
 - 2.5.2 Software Requirements
- 2.6 Cost Benefit Analysis
- 2.7 Risk Analysis

3. DESIGN

3.1 Methodology

3.2 ER Diagram

3.3 Context Diagram

3.4 Data Flow Diagram

3.5 Use case Diagram

3.6 Database Tables

4. FINAL OUTCOME

5.COST ESTIMATE AND LIMITATION

5.1 Estimate Cost

5.2 Limitation

5.3 Maintenance Cost

6. CONCLUSION

7.REFERENCE

CHAPTER- 01

INTRODUCTION

1.1 Project background

The use of chatbots has become increasingly prevalent in recent years, revolutionizing the way businesses interact with customers and users. Chatbots are computer programs designed to simulate human conversation and provide automated responses. They have the potential to streamline communication processes, improve customer service, and provide quick access to information.

The motivation behind this simple chatbot project stems from the need to develop a user-friendly and efficient conversational agent that can cater to various user queries and engage in natural language interactions. Many existing chatbot systems are complex and require extensive resources for development and maintenance. This project aims to address this issue by creating a simplified chatbot solution that can be easily implemented and customized.

overall, the simple chatbot project aims to bridge the gap between users and information by providing an accessible and intuitive conversational interface. By creating a user-friendly and efficient chatbot, this project aims to enhance user experiences, streamline communication processes, and contribute to the growing field of conversational AI.

1.2 Scope of the Project

The scope of a simple chatbot project can vary depending on the specific goals and requirements of the project. However, here are some key aspects that typically fall within the scope of such a project:

1. Functionalities: The chatbot should be designed to handle a range of user queries and provide appropriate responses. The specific functionalities and domains covered by the chatbot will depend on the project's objectives. It could be a customer support chatbot, an informational chatbot, or a personal assistant chatbot, among others.

2. Natural Language Understanding: The chatbot should be capable of understanding user inputs and extracting relevant information. This involves implementing NLP techniques, such as intent recognition, entity extraction, and sentiment analysis, to comprehend user queries effectively.

3. Dialogue Management: The chatbot should maintain context and manage the flow of conversation to create a coherent and engaging dialogue.

4. User Interaction: The chatbot should provide a user-friendly and intuitive interface for users to interact with. This may involve implementing features such as text-based or voice-based input/output, error handling, and user prompts to guide the conversation.

5. Response Generation: The chatbot should generate appropriate response-based approaches to generate natural and contextually relevant replies.

CHAPTER-02

ANALYSIS

2.1 Existing system and proposed system:

The existing system for a simple chatbot may involve a basic rule-based approach where predefined patterns and responses are programmed into the chatbot. This type of chatbot uses if-else statements or pattern matching to identify user inputs and provide corresponding predefined responses. The existing system may have limited capabilities and lacks the ability to understand context or handle complex conversations.

2.1.1 History and Evolution of the system:

Lack of understanding: Simple chatbots often struggle to understand context and meaning beyond the literal interpretation of words. They rely heavily on keyword matching and predefined responses, which can lead to inaccurate or irrelevant answers.

Limited responses: Due to their fixed response patterns, simple chatbots have a restricted set of preprogrammed responses. They may not be able to handle complex or unexpected queries, resulting in generic or unhelpful replies.

Inability to learn and adapt: Simple chatbots typically lack the ability to learn from user interactions and improve over time. They don't have a memory of previous conversations, so they cannot provide personalized responses or maintain continuity in discussions.

Poor handling of ambiguity and errors: When faced with ambiguous or unclear queries, simple chatbots often struggle to provide accurate responses. They may either give incorrect answers or request clarification in a repetitive or confusing manner.

Lack of emotional intelligence: Simple chatbots are typically devoid of emotional understanding and empathy. They cannot recognize or respond appropriately to the emotional state of the user, which can result in unsatisfactory or insensitive interactions.

Difficulty with complex tasks: Simple chatbots are designed for straightforward tasks and may not be equipped to handle complex processes or multi-step interactions. They lack the ability to perform actions beyond basic information retrieval or predefined tasks.

Limited language support: Depending on their design and programming, simple chatbots may only support a specific language or have limited multilingual capabilities. This restricts their accessibility and effectiveness for users who speak different languages.

Security and privacy concerns: Simple chatbots may not have robust security measures in place, making them vulnerable to potential data breaches or malicious use. As chatbots can collect user information, there is a need to ensure data privacy and handle sensitive data appropriately.

These issues highlight the limitations of simple chatbots and emphasize the need for more advanced conversational AI systems that can overcome these challenges and provide more natural and intelligent interactions.

2.1.2 Proposed System:

The proposed system for a simple chatbot could utilize a more advanced approach such as a retrieval-based or generative model.

1.Retrieval-Based Chatbot:

In a retrieval-based system, a collection of predefined responses or templates are created based on expected user inputs. The chatbot matches the user's input to a set of predefined patterns or keywords and retrieves the most appropriate response from the predefined responses. This approach can handle more varied conversations and provide relevant responses based on predefined knowledge.

2.Generative Chatbot:

A generative chatbot employs more advanced techniques such as natural language understanding (NLU) and natural language generation (NLG). It utilizes machine learning algorithms like sequence-to-sequence models or transformer models to understand the user's input, generate context-aware responses, and engage in more human-like conversations. Generative chatbots can be trained on large datasets and can learn from the input-output patterns to generate responses that are not preprogrammed.

Both retrieval-based and generative chatbots can benefit from machine learning and artificial intelligence techniques to improve their performance over time. They can be trained using supervised learning, reinforcement learning, or a combination of both to enhance their understanding, response generation, and ability to handle complex queries.

The proposed system can also include additional features like sentiment analysis to understand the user's emotions, entity recognition to extract relevant information, and integration with external APIs to provide dynamic and up-to-date responses.

Overall, the proposed system aims to provide a more intelligent and interactive chatbot that can understand and respond to user queries effectively, leading to a better user experience.

2.2 Feasibility Study:

A feasibility study for a simple chatbot involves assessing the viability and practicality of implementing and operating the chatbot system. Here are some key factors to consider in the feasibility study:

2.2.1 Technical Feasibility:

Evaluate the technical aspects of developing and deploying the chatbot. Consider factors such as the required infrastructure, programming languages or frameworks, availability of necessary APIs or libraries, and compatibility with existing systems. Ensure that the required technology stack is accessible and feasible for the development team.

2.2.2 Economic Feasibility:

Simple chatbots are often considered economically feasible due to their relatively low development and maintenance costs compared to more advanced conversational AI systems. Here are some factors that contribute to their economic feasibility:

Development costs: Simple chatbots can be developed using basic programming frameworks and tools, requiring fewer resources and expertise compared to more complex AI systems. This reduces the upfront development costs, making them more affordable for businesses and individuals.

Maintenance and updates: Simple chatbots typically have less intricate functionality and fewer dependencies on external systems. This means that maintenance and updates are relatively straightforward and require less effort and resources compared to more advanced AI systems.

Scalability: Simple chatbots can handle a significant number of user interactions simultaneously without significant performance issues. This scalability makes them suitable for businesses with high volumes of customer queries, reducing the need for additional human support.

Cost savings in customer support: By automating routine and repetitive customer support tasks, simple chatbots can reduce the need for human agents and lower labor costs. Businesses can

redirect their human resources to more complex and value-added activities, resulting in potential cost savings.

24/7 availability: Simple chatbots can operate round the clock, providing continuous customer support without requiring additional staff for night shifts or overtime. This availability can lead to improved customer satisfaction and potentially increased revenue.

Efficiency gains: Chatbots can provide quick and accurate responses to common customer queries, reducing the time and effort required by human agents. This efficiency gain can improve overall productivity and customer service, resulting in potential cost savings and increased customer loyalty.

However, it's important to note that while simple chatbots may offer economic feasibility in certain scenarios, they may not be suitable for all use cases. Complex or industry-specific requirements may necessitate more sophisticated AI systems, which can involve higher development and maintenance costs. Additionally, the economic feasibility of a chatbot should be evaluated based on the specific business context, expected return on investment, and long-term sustainability.

2.2.3 Simple chatbots generally exhibit good operational feasibility due to the following reasons:

Easy deployment: Simple chatbots are relatively easy to deploy across various platforms and channels. They can be integrated into websites, messaging apps, social media platforms, and other customer touchpoints with minimal technical complexities.

User-friendly interface: Simple chatbots typically have a straightforward user interface, making them easy for both businesses and customers to interact with. Users can engage in conversations by typing text or selecting options from predefined menus, requiring little to no learning curve.

Quick response time: Simple chatbots can provide instant responses to user inquiries, eliminating the need for customers to wait for human assistance. This quick response time enhances the user experience and improves overall efficiency in resolving queries.

24/7 availability: Simple chatbots can operate continuously, providing round-the-clock support to customers. They do not require breaks or rest, ensuring consistent availability and responsiveness regardless of time zones or business hours.

Scalability: Simple chatbots can handle a high volume of user interactions simultaneously. As the user base grows, businesses can easily scale up their chatbot infrastructure to accommodate increased demand without compromising performance or incurring significant additional costs.

Standardized processes: Simple chatbots can assist with routine and standardized tasks, such as answering frequently asked questions, providing basic information, or collecting user data. By automating these processes, businesses can ensure consistent and error-free execution.

Reduction in human errors: Simple chatbots minimize the chances of human errors that can occur during customer interactions. They follow predefined rules and responses, reducing the likelihood of inaccuracies, miscommunication, or inconsistencies in information provided.

Enhanced productivity: By handling repetitive and low-level inquiries, simple chatbots free up human resources to focus on more complex and critical tasks. This improves overall productivity and efficiency within the organization.

While simple chatbots offer operational feasibility in many scenarios, their limitations should also be considered. Complex or specialized tasks may require human intervention or more advanced AI systems. It is essential to evaluate the specific operational requirements and user expectations to ensure that a simple chatbot aligns with the organization's objectives and delivers the desired outcomes.

2.3 Nonfunctional Requirements:

1. **Reliability:** The chatbot should be available and responsive at least 99% of the time.
2. **Performance:** The chatbot should respond to user queries within 2 seconds.
3. **Scalability:** The chatbot should be able to handle a growing number of concurrent users without significant performance degradation.
4. **Security:** The chatbot should implement secure communication protocols and protect user data and privacy.
5. **Usability:** The chatbot should have an intuitive and user-friendly interface, making it easy for users to interact with.

2.3.1 Product Requirements:

1. **Natural Language Processing:** The chatbot should be capable of understanding and interpreting user input in natural language.

- 2. Response Generation:** The chatbot should generate accurate and relevant responses to user queries.
- 3. Contextual Understanding:** The chatbot should maintain context throughout the conversation and provide contextually appropriate responses.
- 4. Integration:** The chatbot should integrate with other systems or APIs to fetch relevant information or perform actions on behalf of the user.
- 5. Multilingual Support:** The chatbot should support multiple languages to cater to a diverse user base.

2.3.2 Organizational Requirements:

- 1. Maintenance and Support:** The organization should provide regular maintenance and support for the chatbot, including bug fixes and performance improvements.
- 2. Data Privacy and Compliance:** The organization should ensure that the chatbot complies with data protection regulations and handles user data securely.
- 3. Training and Updates:** The organization should invest in continuous training and improvement of the chatbot's capabilities, keeping it up to date with the latest technologies and user expectations.
- 4. Collaboration:** The organization should foster collaboration between development teams, subject matter experts, and stakeholders to ensure the chatbot meets the organization's goals and requirements.
- 5. Documentation:** The organization should maintain comprehensive documentation for the chatbot, including user guides, technical specifications, and training materials.

Please note that these requirements can vary depending on the specific needs and goals of the organization and the intended use of the chatbot.

2.4 Functional Requirements

User Input Handling: The chatbot should be able to receive and understand user input, whether it is in the form of text, voice, or other modalities. It should have the capability to process and interpret user queries or commands.

Natural Language Processing (NLP): The chatbot should be equipped with NLP techniques to understand and extract meaning from user input. It should be able to handle variations in language, account for different sentence structures, and extract relevant information.

Response Generation: The chatbot should generate appropriate and relevant responses based on user input. It should have the ability to provide information, answer questions, and engage in meaningful conversations with users.

Intent Recognition: The chatbot should be able to identify the intent behind the user's query or command. By understanding the user's intention, the chatbot can provide more accurate and effective responses.

Contextual Awareness: The chatbot should maintain context throughout the conversation to provide relevant and consistent responses. It should be able to remember previous interactions, user preferences, and any relevant information to improve the quality of the conversation.

Knowledge Base Integration: If the chatbot is designed to provide specific information or support on a particular topic, it should be integrated with a knowledge base or database. This allows the chatbot to retrieve and provide accurate information to the user.

Error Handling: The chatbot should be able to handle and recover from errors or ambiguous user input. It should be designed to provide helpful error messages and guide the user towards providing clearer input if necessary.

Personalization and User Profiles: If the chatbot supports user profiles, it should be able to personalize responses based on the user's preferences, history, or previous interactions. This can enhance the user experience and make the conversation feel more tailored.

Integration with External Systems: Depending on its purpose, the chatbot may need to integrate with external systems or APIs to perform certain tasks or retrieve specific information. This can include accessing databases, making API calls, or connecting to other software platforms.

Multichannel Support: The chatbot should be able to communicate across multiple channels, such as websites, mobile apps, messaging platforms, or voice assistants. It should adapt its responses and interaction style based on the channel it is being used on.

These are some of the common functional requirements for a simple chatbot. The specific requirements may vary based on the intended use case and the complexity of the chatbot's functionalities.

2.5 Project Requirement Specification:

Purpose: The purpose of the chatbot is to provide automated assistance and answer user queries in a conversational manner.

Target Audience: Identify the specific user group or audience for the chatbot. For example, it could be customers of a particular product or service, employees of a company, or general users seeking information.

Functionality: Specify the specific functions and capabilities the chatbot should have. For example, it could include answering frequently asked questions, providing product recommendations, processing transactions, or collecting user feedback.

Integration: Determine if the chatbot needs to integrate with any existing systems, such as customer relationship management (CRM) software, databases, or external APIs. Specify the integration requirements and any data exchange formats needed.

Language and Platform: Define the programming language and platform preferences for developing the chatbot. Consider factors such as the target platform (web, mobile, messaging apps) and the development team's expertise.

Security and Privacy: Identify any security or privacy requirements for the chatbot. Determine how sensitive user information will be handled and stored and ensure compliance with relevant data protection regulations.

2.5.1 Hardware Requirements:

Server Infrastructure: Determine the hardware resources required to host the chatbot. Consider factors such as server capacity, processing power, memory, and storage based on the expected workload and user traffic.

Networking: Ensure that the server infrastructure has a reliable and stable network connection to handle incoming user requests and responses.

2.5.2 Software Requirements:

Programming Language: Choose a programming language suitable for developing the chatbot. Popular options include Python, Java, Node.js, or Ruby.

Development Framework: Select a chatbot development framework that provides the necessary tools and libraries for building conversational interfaces. Examples include Dialog flow, Microsoft Bot Framework, or Rasa.

Natural Language Processing (NLP): Integrate an NLP library or service to enable the chatbot to understand and interpret user inputs accurately. Common NLP frameworks include Natural Language Toolkit (NLTK), spacey, or Google Cloud Natural Language Processing API.

Integration APIs: If the chatbot needs to interact with external systems or retrieve data from databases, identify the relevant APIs and libraries required for integration.

Hosting Platform: Choose a hosting platform to deploy the chatbot, such as cloud-based services like Amazon Web Services (AWS), Google Cloud Platform (GCP), or Microsoft Azure. Consider factors such as scalability, reliability, and ease of deployment.

User Interface: Decide on the type of user interface for the chatbot, such as a web-based interface, a mobile app, or integration with messaging platforms like Facebook Messenger or Slack.

These requirements provide a general overview, and you can tailor them further based on the specific needs and objectives of your simple chatbot project.

2.6 Cost-benefit analysis:

A cost-benefit analysis (CBA) is a useful tool for evaluating the potential advantages and disadvantages of implementing a simple chatbot. Let's break down the analysis into two main aspects:

Costs:

a. Development: This includes the initial investment to develop and design the chatbot, which involves hiring developers or allocating resources from your development team. Costs may also include acquiring necessary software or platforms.

b. Maintenance: Chatbots require ongoing maintenance, updates, and bug fixes. These costs involve developer time, infrastructure maintenance, and software updates.

c. Integration: Depending on the complexity of the chatbot and the systems it needs to interact with, integration costs may arise. This includes integrating the chatbot with existing databases, APIs, or third-party platforms.

d. Training and support: If your chatbot needs to be trained with specific data or requires ongoing monitoring, training costs or the need for support staff should be considered.

Benefits:

a. Automation and efficiency: A chatbot can automate repetitive tasks, answer common questions, and assist with customer support. This can save time for both users and support staff, leading to increased operational efficiency.

b. 24/7 availability: Unlike human agents, a chatbot can be available 24/7, providing support and information to users at any time. This can enhance customer satisfaction and reduce response time.

c. Scalability: Chatbots can handle multiple conversations simultaneously, making them scalable for businesses with high customer volumes. They can handle many inquiries without experiencing fatigue or delays.

d. Cost savings: By reducing the need for human resources in certain tasks, chatbots can potentially lead to cost savings in the long run.

To conduct a thorough cost-benefit analysis, you'll need to assign monetary values or estimates to each cost and benefit mentioned above. This can be done by considering factors such as labor costs, customer acquisition costs, average handling times, customer satisfaction metrics, and potential revenue increases.

It's important to note that the effectiveness and return on investment of a chatbot will vary depending on the specific context, industry, and business objectives. Conducting a detailed analysis and monitoring the chatbot's performance after implementation will help in making data-driven decisions about its continued use and improvement.

2.7 Risk Analysis:

When conducting a risk analysis for a simple chatbot, it's important to consider potential risks and vulnerabilities associated with its design, functionality, and deployment. Here are some key areas to assess:

Privacy and data security: Evaluate how the chatbot handles user data, including personally identifiable information (PII). Assess the measures in place to protect data confidentiality, integrity, and availability. Consider potential risks such as data breaches or unauthorized access.

Accuracy and reliability: Analyse the chatbot's performance in understanding and responding to user queries. Assess the risk of providing inaccurate or misleading information, which could have negative consequences for users relying on the chatbot's responses.

User safety: Identify potential risks that may arise from the chatbot's interactions with users. Evaluate the system's ability to handle sensitive or inappropriate content, prevent harm, and escalate to human intervention when necessary. Consider risks associated with discriminatory or biased responses.

Ethical considerations: Assess the ethical implications of the chatbot's design and behaviour. Analyse potential biases in data used for training, potential reinforcement of harmful behaviour, or manipulation of users. Consider the chatbot's transparency and disclosure of being an AI-driven system.

System vulnerabilities: Identify potential technical vulnerabilities in the chatbot's implementation. Evaluate the security of the underlying infrastructure, communication protocols, and external integrations. Assess risks such as code injection, denial-of-service attacks, or unauthorized system access.

Scalability and performance: Consider the chatbot's ability to handle increasing user demand. Assess the risk of system overload, degradation of response times, or potential downtime during peak usage periods.

Compliance and legal requirements: Ensure the chatbot complies with relevant regulations and legal requirements, such as data protection laws or industry-specific guidelines. Identify potential risks associated with non-compliance or legal liabilities.

User feedback and adaptation: Establish mechanisms to collect user feedback and monitor the chatbot's performance over time. Assess risks associated with feedback loops, biased feedback, or over-reliance on user suggestions, which could lead to the system reinforcing incorrect or harmful behaviour.

Once you've identified potential risks, you can prioritize them based on their impact and likelihood. This will help you develop mitigation strategies, such as implementing security measures, conducting regular audits, and providing clear guidelines for human oversight and intervention when needed.

CHAPTER-03

DESIGN

3.1 Methodology:

The methodology of a simple chatbot typically involves the following steps:

There are several software development life cycle (SDLC) models that can be used to create a simple chatbot. One commonly used model is the Agile methodology, specifically the Scrum framework.

Here's a brief overview of how you could apply the Agile/Scrum approach to develop a chatbot:

Define the purpose and scope: Determine the specific purpose of the chatbot and the tasks it will perform. This could include providing information, answering frequently asked questions, or assisting with basic tasks.

Choose a platform or framework: Select a platform or framework to build your chatbot. There are various options available, including using existing chatbot development frameworks, such as Dialog flow, and Microsoft Bot Framework, or creating a custom chatbot using programming languages like Python.

Design the conversation flow: Plan the flow of the conversation between the chatbot and the user. Identify the possible user inputs and define the appropriate responses based on those inputs. You can use techniques like decision trees, state machines, or natural language processing (NLP) to handle user interactions effectively.

Collect and preprocess data: If your chatbot needs to understand and respond to user queries, you may need to collect and preprocess data to train your chatbot. This can involve creating a dataset of user queries and their corresponding responses or using existing datasets.

Train the chatbot (if applicable): If your chatbot uses machine learning or NLP techniques, you'll need to train it using the collected data. This step involves feeding the data into a machine-learning model and optimizing its parameters to improve the chatbot's performance.

Implement the chatbot: Develop the chatbot using your chosen platform or framework. This involves writing the necessary code to handle user inputs, process queries, and generate appropriate responses. You may also need to integrate external APIs or databases to fetch relevant information.

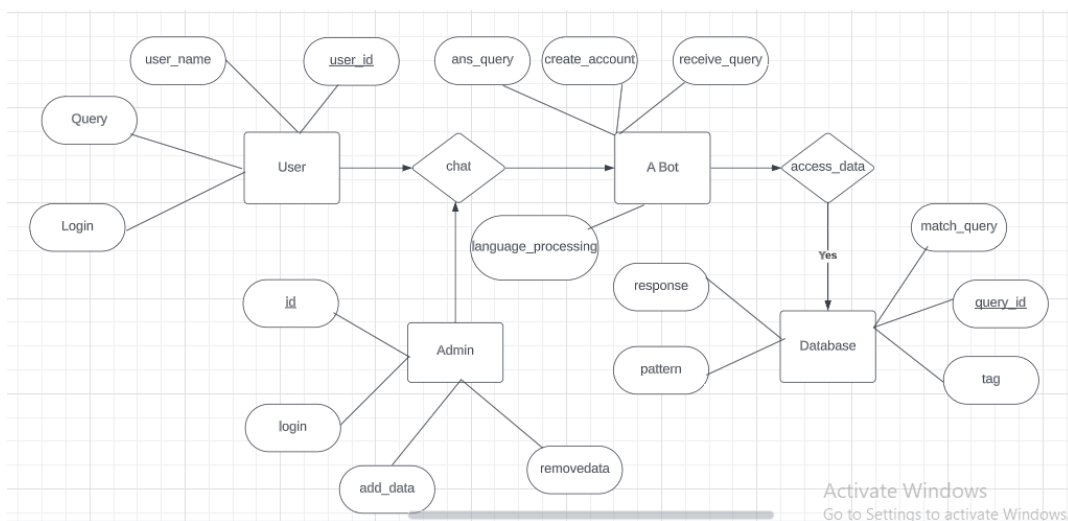
Test and refine: Test the chatbot thoroughly to ensure it performs as expected. Identify any issues or limitations and refine the chatbot's behaviour and responses accordingly. Iterate on the design and implementation to improve the user experience.

Deploy the chatbot: Once you are satisfied with the chatbot's performance, deploy it to a suitable platform or integrate it into your desired system. Ensure that it is accessible and available to users.

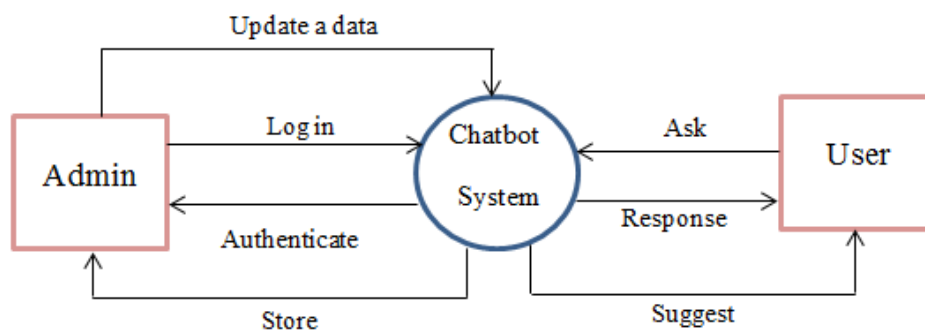
Monitor and maintain: Regularly monitor the chatbot's performance and user feedback. Continuously improve and update the chatbot based on user interactions, new requirements, or emerging trends.

It's important to note that the complexity and sophistication of a chatbot can vary greatly depending on its purpose and the resources available for its development. The above methodology outlines a general approach to building a simple chatbot, and additional steps or techniques may be required for more advanced functionalities.

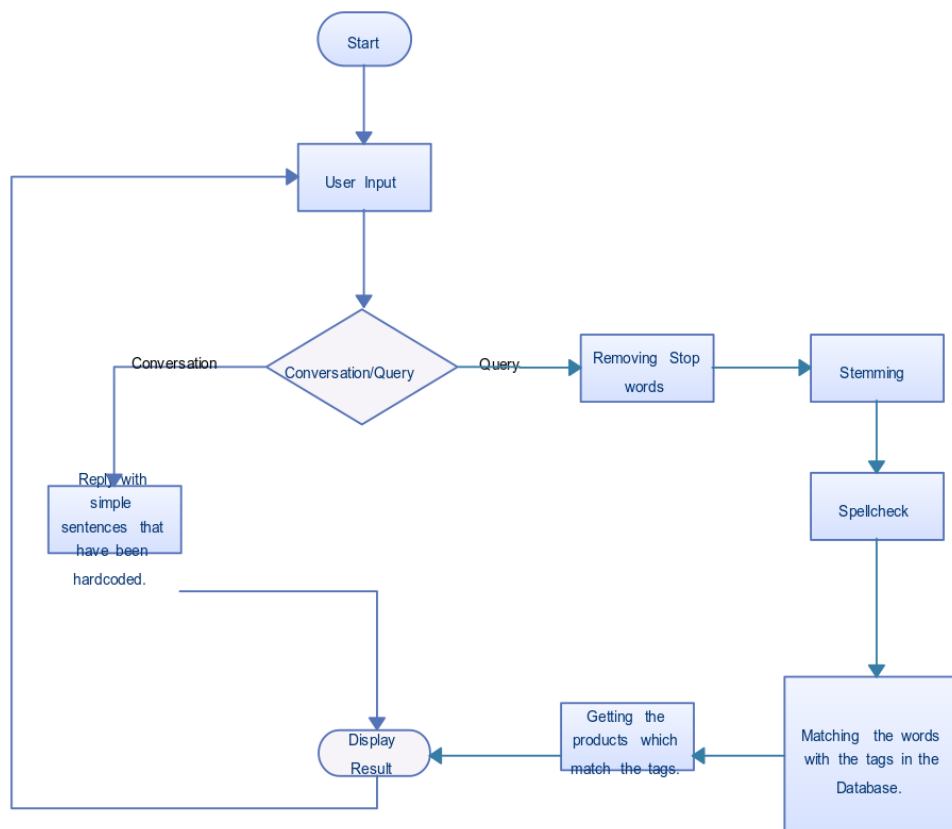
3.2 ER Diagram:



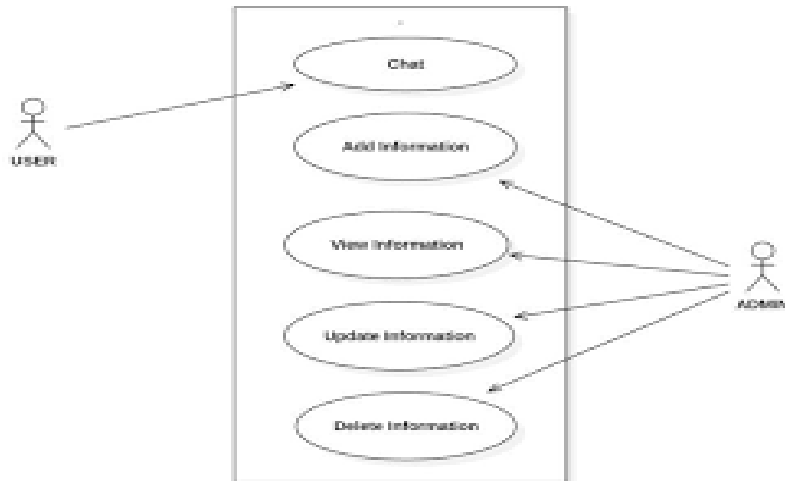
3.3 Context Diagram:



3.4 Data Flow Diagram:



3.5 Use case Diagram:



3.6 Database Tables:

```

intents.json - D:\datafair projects\final chatbot\intents.json (3.6.0)
File Edit Format Run Options Window Help
{"intents": [
  {
    "tag": "greeting",
    "patterns": ["Hi there", "How are you", "Is anyone there?", "Hey", "Hola", "Hello", "Good day"],
    "responses": ["Hello, thanks for asking", "Good to see you again", "Hi there, how can I help?"],
    "context": [""]
  },
  {
    "tag": "goodbye",
    "patterns": ["Bye", "See you later", "Goodbye", "Nice chatting to you, bye", "Till next time"],
    "responses": ["See you!", "Have a nice day", "Bye! Come back again soon."],
    "context": [""]
  },
  {
    "tag": "thanks",
    "patterns": ["Thanks", "Thank you", "That's helpful", "Awesome, thanks", "Thanks for helping m"],
    "responses": ["Happy to help!", "Any time!", "My pleasure"],
    "context": [""]
  },
  {
    "tag": "noanswer",
    "patterns": [],
    "responses": ["Sorry, can't understand you", "Please give me more info", "Not sure I understand"],
    "context": [""]
  },
  {
    "tag": "options",
    "patterns": ["How you could help me?", "What you can do?", "What help you provide?", "How you"],
    "responses": ["I can guide you through Adverse drug reaction list, Blood pressure tracking, Ho"],
    "context": [""]
  },
  {
    "tag": "adverse_drug",
    "patterns": ["How to check Adverse drug reaction?", "Open adverse drugs module", "Give me a li"],
    "responses": ["Navigating to Adverse drug reaction module"],
    "context": [""]
  }
]
}
Ln: 1 Col: 0

```

CHAPTER-04

FINAL OUTCOME

The outcome of a simple chatbot can vary depending on the specific goals and capabilities of the bot. However, here are some common features and characteristics that you might expect from a basic chatbot:

Natural Language Processing (NLP): The chatbot should be able to understand and interpret user input in natural language. It should be equipped with NLP algorithms to extract meaning from user messages.

Predefined Responses: The chatbot can be programmed with a set of predefined responses to common user inquiries or prompts. These responses are typically created based on anticipated user questions or actions.

Conversation Flow: The chatbot should be able to engage in a coherent conversation with users, maintaining context and understanding the flow of conversation. It can use techniques like context tracking or state management to ensure continuity.

Information Retrieval: The chatbot may be designed to retrieve information from a knowledge base or external sources. It can provide answers to frequently asked questions or deliver relevant data based on user queries.

Personalization: A more advanced chatbot might have the ability to personalize responses based on user preferences or historical interactions. It can remember user information and tailor the conversation accordingly.

Integration with Systems: The chatbot can integrate with other systems or services to perform tasks on behalf of the user. For example, it could place an order, book a reservation, or fetch information from a database.

Error Handling: The chatbot should be equipped with error handling mechanisms to handle misunderstandings or ambiguous queries. It can provide clarifications, suggest alternative phrasing, or escalate the issue to a human operator if necessary.

CHAPTER-05

COST ESTIMATE AND LIMITATION

5.1 Estimate Cost:

The cost of a simple chatbot can vary depending on several factors, including the complexity of the chatbot's functionality, the development platform used, the desired integration with external systems, and the level of customization required. Here are some cost considerations for developing a simple chatbot:

Development Platform: There are various development platforms available for building chatbots, ranging from free options to commercial solutions. Some popular platforms include Dialogflow, IBM Watson Assistant, Microsoft Bot Framework, and custom-built solutions. Free platforms may have limitations in terms of features and scalability, while commercial platforms often offer more advanced capabilities at a cost. Costs for commercial platforms can vary, but they usually involve a monthly or annual subscription fee.

Development Effort: The complexity and functionality of the chatbot will influence the development effort required. A simple chatbot with basic responses and limited integrations will require less development effort compared to a chatbot with advanced natural language processing, machine learning, or integration with external systems. Development effort can vary widely, ranging from a few hours to several weeks or more.

Integration and Customization: If you need the chatbot to integrate with external systems or APIs, such as databases, CRM systems, or other applications, it may require additional development effort and potentially incur additional costs. Similarly, if you require a high level of customization in terms of the chatbot's appearance, user interface, or conversational flow, it may increase the development effort and cost.

Hosting and Infrastructure: You will also need to consider the hosting and infrastructure costs for deploying the chatbot. Cloud-based hosting services like Amazon Web Services (AWS) or Microsoft Azure may charge based on factors such as the number of requests or the computing resources used.

Maintenance and Updates: Once the chatbot is deployed, there may be ongoing maintenance and updates required. This can involve fixing bugs, improving performance, adding new features, or updating integrations. The cost of maintenance and updates will depend on the complexity of the chatbot and the frequency of changes.

Given these factors, it is difficult to provide an exact cost without specific requirements. However, for a simple chatbot without extensive customization or integrations, the development cost could range from a few hundred dollars to a few thousand dollars. Ongoing costs for hosting, maintenance, and updates may range from tens to hundreds of dollars per month, depending on the scale and complexity of the chatbot.

5.2 Limitation:

A simple chatbot has several limitations, including:

Lack of understanding context: Simple chatbots typically rely on predefined rules and patterns to generate responses. They struggle to understand the context of a conversation beyond the specific keywords or phrases they are programmed to recognize. As a result, they may provide irrelevant or inaccurate responses.

Inability to handle complex queries: Simple chatbots often struggle with handling complex or nuanced queries. They may not be able to comprehend intricate questions or provide detailed and accurate answers to them. Their limited capabilities restrict them to basic, straightforward interactions.

Limited learning ability: Simple chatbots lack advanced machine learning capabilities. They do not have the ability to learn and improve over time based on user interactions. They are typically programmed with fixed responses and cannot adapt or evolve their behaviour based on user feedback or changing requirements.

Difficulty in handling ambiguity: Chatbot algorithms used in simple chatbots may find it challenging to handle ambiguous queries or statements. They typically lack the ability to ask clarifying questions or seek additional information when faced with unclear user inputs. Consequently, they may provide incorrect or nonsensical responses in such situations.

Lack of emotional intelligence: Simple chatbots often struggle to understand or respond to emotions expressed by users. They are unable to empathize, gauge the emotional state of the user, or adjust their responses accordingly. This limitation can lead to inappropriate or inadequate interactions, especially in situations where emotional support or understanding is required.

Dependency on predefined responses: Simple chatbots rely on pre-programmed responses and patterns. They do not have the ability to generate novel or creative responses to unique user inputs. This lack of flexibility can make interactions with simple chatbots feel rigid and impersonal.

Domain-specific limitations: Simple chatbots are typically designed to operate within specific domains or industries. They may lack knowledge or expertise outside their designated area. When faced with queries or topics beyond their scope, they may struggle to provide meaningful responses or resort to generic replies.

Difficulty in natural language understanding: Simple chatbots may struggle to understand and interpret natural language in a conversational context. They may misinterpret sentence structures, idioms, sarcasm, or other forms of linguistic nuance. This limitation can lead to miscommunication and frustration for users.

Overall, while simple chatbots can handle basic inquiries and tasks, they are limited in their ability to provide sophisticated, context-aware, and emotionally intelligent interactions. Advanced chatbot models with natural language understanding and machine learning capabilities have been developed to address these limitations and offer more robust conversational experiences.

5.3 Maintenance Cost:

The maintenance cost of a simple chatbot can vary depending on several factors, such as the complexity of the chatbot, the platform on which it is deployed, and the desired level of support and updates. Here are some components that may contribute to the maintenance cost:

Development and Deployment: The initial development and deployment of a chatbot can involve costs related to software development, integration with existing systems, and hosting infrastructure. This cost is typically a one-time expense.

Hosting and Infrastructure: Chatbots require hosting infrastructure to remain operational. The cost will depend on the chosen hosting solution, such as cloud-based platforms like Amazon Web Services (AWS) or Microsoft Azure. The hosting cost can be a recurring monthly or annual expense.

Updates and Enhancements: Chatbots may require ongoing updates and enhancements to improve performance, add new features, or address any issues or bugs. The cost will depend on the frequency and complexity of the updates.

Natural Language Processing (NLP) Services: If the chatbot incorporates NLP capabilities to understand and process user input, there might be additional costs associated with using NLP services or APIs, such as those provided by Google Cloud Natural Language Processing, IBM Watson, or Microsoft Azure Cognitive Services.

Maintenance and Support: Chatbots require ongoing maintenance and support to ensure they remain operational and perform as expected. This can include monitoring the chatbot's performance, handling user feedback and inquiries, and making necessary adjustments. The cost can vary depending on the level of support required, such as regular maintenance or 24/7 support.

Integration with External Systems: If the chatbot needs to integrate with external systems or databases, there may be costs associated with maintaining those integrations, such as API access fees or development efforts.

It's important to note that the maintenance cost of a simple chatbot is typically lower compared to more complex and advanced chatbot systems. The above factors should be considered when estimating the overall maintenance cost, but it's advisable to consult with chatbot developers or service providers to get a more accurate assessment based on your specific requirements.

CHAPTER-06

CONCLUSION

In conclusion, a simple chatbot can be an effective tool for engaging with users and providing basic information or assistance. While it may not possess the advanced capabilities of more sophisticated chatbots, it can still offer value by answering frequently asked questions, providing relevant resources, or offering simple recommendations.

The primary advantages of a simple chatbot are its ease of implementation and low cost compared to more complex solutions. It can be quickly developed using pre-built platforms or frameworks, and its straightforward functionality allows users to interact with it without any technical expertise.

However, it's important to acknowledge the limitations of a simple chatbot. It may struggle to understand complex or nuanced queries, and its responses may be limited in scope. Additionally, it may lack the ability to learn and improve over time, as it typically relies on predefined rules or keyword matching.

Overall, a simple chatbot can be a valuable addition to a website, application, or customer support system, particularly for handling routine inquiries and providing basic assistance. It can enhance user experience, save time, and streamline processes, but it's essential to set clear expectations and ensure that users understand its capabilities to avoid frustration.

Reference:

Google Search Engine

Google Scholar

ResearchGate

YouTube

ChatGPT